Study and Implementation of Git

Ambrish Singh
Department of Computer Science
ambrish.mitmpl2026@learner.manipal.edu

Abstract – This report documents an effort to build a working understanding of Git [2], the version control system used throughout the exercise. The interactive Learn Git Branching platform [1] was used to explore core ideas such as commits, branches, merging, rebasing, and remote repositories, and these ideas were then applied directly on a sample repository holding the assignment's solution files.

Keywords—branching, commits, Git, merging, rebasing, remote repositories, version control

I. INTRODUCTION

Software evolves constantly during development, which makes it essential to keep track of earlier versions of a project as work progresses. Version control systems address this need by giving teams a structured way to record changes and preserve a project's history over time. Git is the version control system most widely adopted by developers worldwide. Unlike centralized systems, Git is distributed: it keeps a full repository on the local machine, so most operations can be carried out without a constant connection to a remote server. Core Git features such as cloning, committing, pushing, pulling, branching, merging, and rebasing all work together to manage that project history. This report works through the internal structures behind these commands and explains how each one actually behaves.

The remainder of this report is organized in two parts. Sections II through VII cover the underlying concepts: how a commit is stored, what a branch actually points to, how merging differs from rebasing, how push and pull interact with a remote repository, how git log differs from git reflog, and how merge conflicts and stashing are handled. Section VIII then puts these ideas into practice, first through the Main and Remote modules on the Learn Git Branching website, and afterward on a local repository created for this task, where the Task 1 Python solutions were staged, committed, and pushed.

II. GIT COMMITS AND HISTORY

A. *GIT Commits*

A commit is essentially a snapshot of the project frozen at one moment in time. Every commit stores a reference back to the commit that came before it, and that chain of references is how Git reconstructs the full history of a project. Each time a new commit is made, the branch currently checked out simply advances to point at it, and because the new commit permanently records the hash of its parent, none of the earlier snapshots are ever lost.

B. *HEAD and Relative References*

HEAD is Git's name for whichever branch is currently checked out, and it usually points at that branch's most recent commit.

Rather than always typing out a full commit hash, such as 7af68a4128b936d85a11cda34a9e22303fb1462d, Git lets earlier commits be reached through relative references built off `HEAD`, such as `HEAD^` (the parent commit) or `HEAD~1` (the commit one step back). These shorthand references make it far easier to navigate backward through a project's history.

III. BRANCHING AND BRANCH MANAGEMENT

A. *What a Branch really is*

A branch such as `main` or `feature` is not a duplicate copy of the entire project; it is nothing more than a small file inside `.git` holding the hash of a single commit. Creating a new branch, whether with `git branch` or `git checkout -b <branch name>`, is therefore almost instantaneous, since it only has to write one new file containing that hash. In effect, a branch is just a movable pointer to a commit, and making a new commit on that branch simply pushes the pointer forward to the newest snapshot.

B. *Merging*

Merging is how Git brings the changes from one branch into another. When the branch being merged into has not advanced since the other branch branched off, Git performs a fast-forward merge: it simply slides the pointer forward, and no additional commit is generated. When both branches have moved on with commits of their own, Git instead performs a three-way merge — it locates the last commit the two branches share, compares what changed on each side since then, and produces a single new commit with two parents that ties both histories together.

C. *Rebasing*

Rebasing offers a second way to bring one branch's changes into another, though it works quite differently from merging. Running `git rebase main`, for example, takes every commit unique to the current branch and replays it one by one on top of main's latest commit. Because each replayed commit is technically new, it receives a fresh hash even though its file contents are identical to before, and the end result is a clean, linear history with no merge commit at all. This rewriting of history is exactly why rebasing should never be used on a branch that has already been pushed and is being used by others — doing so leaves two different, conflicting histories of what should be the same changes.

IV. REMOTE REPOSITORIES : PUSH & PULL

A. *Tracking branching and origin*

A remote — conventionally named `origin` — is simply a shorthand label pointing at a repository hosted

elsewhere, usually on the internet. Alongside it, Git keeps a set of remote-tracking references, such as `origin/main`, that record where each branch stood the last time the local repository communicated with that remote. These tracking references are only refreshed by a fetch, pull, or push; making an ordinary local commit has no effect on them at all.

B. Fetch vs Pull

`git fetch` retrieves new commits from the remote and updates references such as `origin/main`, but it never touches the files in the working directory — which is exactly why it is considered a safe command to run at any time. `git pull`, by contrast, is effectively a fetch immediately followed by a merge of the newly updated remote branch into the current one. Because that merge step is built in, a pull can produce a merge commit of its own, and it can run into the same kind of conflict an ordinary merge would.

C. Force Pushing

An ordinary git push succeeds only when it is adding new commits cleanly on top of what the remote already holds. Once a local branch has been rebased, or an existing commit has been amended, the local and remote histories diverge, and a plain push is rejected outright rather than doing anything at all.

`git push --force` overrides that safeguard and overwrites the remote branch regardless, but because it can erase commits that a collaborator has already pulled down, it needs to be used with real caution.

V. History, Logs and the Reference Log

A. git log and Commit Ancestry

`git log` walks the commit graph outward from HEAD, moving backward through each commit's parent, and so it displays the branch's genuine, permanent history — every commit it lists is one that can actually be reached by following those parent links. Flags such as `--oneline`, `--graph`, and `--all` condense this output and make the overall branching and merge structure much easier to read at a glance.

B. git reflog

`git reflog` serves a very different purpose from `git log`. It is a purely local log of every position HEAD has occupied, listed in the order those moves actually happened, so it captures checkouts, commits, merges, rebases, and resets alike. Unlike the entries in `git log`, reflog entries need not connect to one another at all, since operations like reset or rebase can strand old commits behind instead of building on top of them. Reflog entries live only on the local machine and are never transmitted to a remote.

- git log traces the ancestry of the current commit.
- git reflog records everywhere HEAD has actually pointed, in the order it happened.

VI. Resolving Merge Conflicts

A merge conflict arises when Git cannot automatically reconcile two branches because each one has modified the exact same line of the exact same file. When that happens, Git halts the merge and marks the disputed section of the file directly, bracketing it with `<<<<<<< HEAD`, a `=======` divider, and `>>>>>>>` followed by the name of the incoming branch.

Resolving a conflict is a manual process: the file must be opened and edited by hand to decide on the final content, after which `git add` marks it as resolved and `git commit` completes the merge.

VII. Stashing

`git stash` temporarily sets aside changes that have not yet been committed, without creating an actual commit for them. A typical scenario is being midway through a feature on `branch1` when an urgent bug forces a switch to `branch2` — Git will normally block that switch because the uncommitted changes would be overwritten. Stashing clears the working directory so the switch can happen, letting the bug be fixed before returning to the stashed work.

`git stash list` displays every stash currently saved, `git stash pop` restores the most recent one and removes it from that list, and `git stash apply` restores it while keeping the copy in place in case it is needed again later.

VIII. Demonstration : LearnGitBranching and a Local Repository

A. A little about the Main and Remote tabs on Learn Git Branching

Working through the Main tab clarified the basic Git commands and how commits and branches are actually managed within a local repository, while the Remote tab built a much clearer picture of pushing, pulling, and working against remote branches. The exercises contrasting merge and rebase were particularly useful, since the two approaches leave visibly different commit histories behind. The diverged-remote exercises stood out as well, especially watching Git reject a push the moment the local and remote histories fell out of sync with one another.

B. Local Repository setup and Task 1 integration

To move from theory into practice, I ran `git init` inside a folder holding the eight Python solution files written for Task 1. Every file was staged in one step with `git add .`, and the whole set was recorded as a single initial commit using `git commit`.

I then created a matching empty repository on GitHub and linked it to the local one as `origin` with `git remote add origin`, before pushing that initial commit up with `git push -u origin main`. Running `git log --oneline` immediately afterward confirmed a single commit at the head of the branch, labeled with the initial commit message and listing all eight newly created solution files.

C. Demonstrating Commit, Branch, and Merge

Next, I created a new branch called `feature-update` with `git checkout -b`, added a small enhancement to the Task 1, Question 1 Python file so that its output dictionary arranges keys alphabetically, and committed that change on the new branch. Switching back to main

and running `git merge feature-update` then completed the merge. Because main had not advanced since feature-update was created, Git carried this out as a fast-forward merge — the branch pointer was simply moved ahead with no separate merge commit, exactly as described in Section III.B. Viewing the result with `git log --oneline --graph --all` afterward showed a single straight line of commits, confirming that no merge commit had been introduced.

D. Demonstrating a Merge Conflict and Resolution

To reproduce a genuine merge conflict, I created a branch named `conflict-branch` and edited the first line of a Task 1 file with a simple demo comment, committing that change on the new branch. The very same line of the same file was then edited differently on main and committed there too, so both branches had diverged on the identical line since their shared ancestor. Running `git merge conflict-branch` from main produced the expected conflict, and Git marked the disputed line with <<<<<<<, =======, and >>>>>>> dividers, clearly separating main's version of the line from the version coming in from conflict-branch.

The file was then edited by hand to remove those markers and keep the intended text, and the conflict was marked resolved with `git add` followed by `git commit`, which completed the merge and added a merge commit with two parents to the project's history.

E. Demonstrating Push and Pull

With the merge complete, I pushed the updated main branch to the remote with `git push origin main`. To then demonstrate a pull, I made a small edit directly on GitHub — a short test comment — and committed it there, so the remote repository held a commit the local copy did not yet have. Running `git pull origin main` fetched that new commit and merged it straight into the local main branch; since local had not diverged from the remote at that point, Git performed this as a fast-forward pull, and the change from GitHub appeared in the local files immediately.

F. Log and Reflog Evidence

Running `git reflog` against the repository listed every checkout, commit, and merge carried out during this demonstration, in the exact order each one happened, with every entry referenced as `HEAD@{n}`. Set alongside the earlier output of `git log --oneline --graph --all`, the two commands illustrate the distinction raised in Section V rather neatly: `git log` shows only the permanent ancestry of commits reachable from the current branch, while `git reflog` shows the complete local movement history of HEAD, including checkouts and other actions that never added any new ancestry to the branch at all.

IX. Conclusion

Working through this task reframed Git, in my own understanding, as more than a list of commands to memorize — it is a coherent system built around commits, branches, and pointers that shift in predictable ways as those commands run. The Learn Git Branching exercises made ideas like branching, merging, and rebasing far easier to visualize, and repeating the same operations on an actual local repository wired up to a real GitHub remote showed how those same ideas play out in practice, giving a clearer sense of how teams actually collaborate on shared code. Deliberately triggering and then resolving a merge conflict was especially valuable, since it demonstrated exactly how Git flags conflicting changes and why they can only be resolved by hand. Altogether, this exercise left Git's underlying behavior, and the everyday workflow built on top of it, considerably clearer than before.

ACKNOWLEDGMENT

I would like to thank the seniors of the Driverless Subsystem for the guidance and resources that made it possible to complete this task with a deeper understanding of Git and the concepts covered in this report.

REFERENCES

[1] Learn Git Branching. [Online]. Available: https://learngitbranching.js.org/

[2] Git, “Git Reference” [Online]. Available: https://git-scm.com/docs